

Data Structure Lab6 : Doubly Linked List 2022-2023

Topics

1. Implement Node Class
2. Implement DoublyLinkedList Class
3. Implement Basic Methods of DoublyLinkedList
 - isEmpty()
 - size()
 - first()
 - last()
 - addFirst()
 - addLast()
 - removeFirst()
 - removeLast()

Homework

1. Describe a method for finding the middle node of a doubly linked list with header and trailer sentinels by “link hopping,” and without relying on explicit knowledge of the size of the list. In the case of an even number of nodes, report the node slightly left of center as the “middle.”

-Initialize two pointers: slow and fast, both starting at the header sentinel.

-Move slow one step forward and fast two steps forward.

-Continue this process until fast reaches the trailer sentinel or fast.next is null.

-When fast stops, slow will be at the middle node. If the list has an even number of nodes, slow will point to the node just left of the center.

Data Structure Lab6 : Doubly Linked List 2022-2023

2. Give an implementation of the size() method for the DoublyLinkedList class, assuming that we did not maintain size as an instance variable.

```
public int size() {  
    int count = 0;  
    Node<T> current = header.next;  
    while (current != trailer) {  
        count++;  
        current = current.next;  
    }  
    return count;  
}
```

3. Implement the equals() method for the DoublyLinkedList class.

```
@Override  
public boolean equals(Object obj) {  
    if (!(obj instanceof DoublyLinkedList)) {  
        return false;  
    }  
    DoublyLinkedList<?> other = (DoublyLinkedList<?>) obj;  
    Node<?> current1 = this.header.next;  
    Node<?> current2 = other.header.next;  
  
    while (current1 != this.trailer && current2 != other.trailer) {  
        if (!current1.data.equals(current2.data)) {  
            return false;  
        }  
        current1 = current1.next;  
        current2 = current2.next;  
    }
```

Data Structure Lab6 : Doubly Linked List 2022-2023

```
    }  
    return current1 == this.trailer && current2 == other.trailer; // Both  
    reached the trailer  
}
```

4. Give an algorithm for concatenating two doubly linked lists L and M, with header and trailer sentinel nodes, into a single list L'.

-If either list L or M is empty, set the new list L' to the non-empty list.

-Connect the last node of L to the first node of M:

- Set L.trailer.prev to M.header.next.
- Set M.header.next.prev to L.trailer.prev.

-Set the trailer of the new list L' to the trailer of M.

-Update the trailer node of L' to point to the appropriate nodes to maintain the structure.

5. Our implementation of a doubly linked list relies on two sentinel nodes, header and trailer, but a single sentinel node that guards both ends of the list should suffice. Reimplement the DoublyLinkedList class using only one sentinel node.

-Create a single sentinel node that points to itself for both next and prev.

-For all operations (add, remove), adjust the pointers accordingly to maintain the doubly linked structure.

-The operations for adding and removing nodes will adjust the next and prev pointers of the sentinel node to point to the new nodes.

6. Implement a circular version of a doubly linked list, without any sentinels, that supports all the public behaviors of the original as well as two new update methods, rotate() and rotateBackward.

Data Structure Lab6 : Doubly Linked List 2022-2023

-Create a node structure where the next pointer of the last node points to the first node and the prev pointer of the first node points to the last node.

-Implement the public methods:

- addFirst(), addLast(), removeFirst(), removeLast(), and others to maintain the circular structure.

7. Implement the clone() method for the DoublyLinkedList class.

```
public DoublyLinkedList<T> clone() {  
    DoublyLinkedList<T> newList = new DoublyLinkedList<>();  
    Node<T> current = this.header.next; // Start from the first real node  
    while (current != this.trailer) {  
        newList.addLast(current.data); // Add each element to the new  
list  
        current = current.next;  
    }  
    return newList;  
}
```